NODE.JS

One-Month Course Material

A beginner-to-intermediate practical roadmap for building backend applications with JavaScript.

Duration: 30 Days **Suggested study:** 1.5–2.5 hours/day

Includes fundamentals • Node runtime • npm • modules • async JavaScript • HTTP • Express • REST APIs • middleware • validation • authentication • databases • testing • deployment • capstone project

Course goal: by the end of the month, you should be able to design and build a small production-style Node.js REST API.

How to Use This Course

This plan assumes you already know basic JavaScript syntax. If JavaScript is new to you, spend extra time on functions, arrays/objects, scope, promises, async/await, and ES modules before moving quickly through Express.

Daily routine

• 20–30 minutes: read the concept notes.

• 30–45 minutes: type the examples yourself instead of copy-pasting.

• 30–60 minutes: complete the practice task.

• 10 minutes: write down what you learned and any errors you encountered.

Tools

• Node.js LTS

• VS Code or another editor

• Terminal / PowerShell

• Git and GitHub

• Postman or another API client

• A database for the capstone (MongoDB or PostgreSQL).

Recommended mindset: use AI as a tutor/debugging assistant, but first try to understand and write the code yourself.

Learning Outcomes

- Explain what Node.js is and how it differs from browser JavaScript.
- Create and manage Node.js projects with npm and package.json.
- Use CommonJS and ES modules appropriately.
- Work with files, paths, environment variables, events, streams, and buffers.
- Understand the event loop, callbacks, promises, and async/await.
- Create HTTP servers and build REST APIs with Express.
- Use routing, middleware, validation, centralized error handling, and status codes.
- Connect an API to a database and perform CRUD operations.
- Implement authentication with hashed passwords and token-based sessions/JWT concepts.
- Write basic unit/integration tests and structure a maintainable backend.
- Prepare a Node.js application for deployment and production-style configuration.

30-Day Roadmap

Week	Focus	Main outcome
1	Node fundamentals	Runtime, npm, modules, files, async, events
2	HTTP + Express	REST API, routing, middleware, validation, errors
3	Database + Auth	CRUD, data modeling, authentication, security
4	Testing + Production	Testing, architecture, logging, deployment, capstone

Week 1 — Node.js Fundamentals

Day 1: What is Node.js?

Learn: Node.js is a JavaScript runtime built around the V8 engine. It lets JavaScript execute outside the browser. Learn runtime vs language, server-side JavaScript, non-blocking I/O, and common use cases.

Practice: Install Node.js LTS. Run node --version and npm --version. Create hello.js and run node hello.js.

Deliverable: A working Node installation and first script.

Day 2: Node REPL and Core Concepts

Learn: Explore the Node REPL, global objects, process, process.env, process.argv, and command-line execution.

Practice: Print process.version, process.argv, and selected environment values. Pass an argument from the terminal.

Deliverable: A small CLI greeting program.

Day 3: npm and package.json

Learn: Learn npm, package.json, dependencies, devDependencies, scripts, semantic versioning, and package-lock.json.

Practice: Initialize a project with npm init. Add a small dependency and create start/dev scripts.

Deliverable: A clean Node project with useful npm scripts.

Day 4: Modules

Learn: Learn module boundaries, exports/imports, CommonJS, ES modules, default/named exports, and when to keep modules small.

Practice: Create math.js, user.js, and app.js. Import functions using the module system configured by your project.

Deliverable: A three-module mini application.

Day 5: File System and Paths

Learn: Learn fs, fs/promises, path, synchronous vs asynchronous file operations, and safe file paths.

Practice: Create a notes CLI that creates, reads, updates, and deletes a JSON file.

Deliverable: A file-based notes manager.

Day 6: Events, Buffers and Streams

Learn: Understand EventEmitter, buffers, readable/writable streams, and why streams are useful for large data.

Practice: Create an EventEmitter example and copy a large text file using streams.

Deliverable: An event + stream practice project.

Day 7: Async JavaScript and Weekly Review

Learn: Review callbacks, promises, async/await, error handling with try/catch, and the Node event loop conceptually.

Practice: Build three versions of a small asynchronous task: callback, promise, and async/await.

Deliverable: Week 1 review + 20-question self-test.

Core Notes — Week 1

Node.js architecture

A Node process runs JavaScript on the server or command line. Node provides APIs for networking, files, processes, streams, cryptography, and more. JavaScript execution is handled by V8, while asynchronous operations are coordinated through Node's runtime mechanisms and event loop.

A useful mental model is: **JavaScript code** → **Node runtime** → **asynchronous I/O** → **callbacks/promises** → **event loop** → **your next JavaScript operation.** The event loop allows a process to keep handling work without blocking on many I/O operations.

npm and package.json

package.json describes a project and its dependencies/scripts. package-lock.json records the resolved dependency tree. Keep production dependencies in dependencies and development-only tools in devDependencies.

```
{
"scripts": {
"start": "node src/server.js",
"dev": "node --watch src/server.js"
}
}
```

Promises and async/await

```
async function loadData() {
try {
const result = await doSomethingAsync();
console.log(result);
} catch (error) {
console.error(error);
}
}
```

Prefer asynchronous APIs for server work when possible. Synchronous file operations can block the event loop and should be used deliberately rather than by default.

Week 2 — HTTP and Express

Day 8: HTTP Fundamentals

Learn: Learn requests, responses, methods (GET/POST/PUT/PATCH/DELETE), headers, body, status codes, JSON, and content types.

Practice: Use Postman to send requests to a public API. Identify method, URL, headers, body, and response.

Deliverable: HTTP request/response worksheet.

Day 9: Native Node HTTP Server

Learn: Create a server with the http module. Learn request URLs, methods, response headers, status codes, and JSON responses.

Practice: Build /, /about, and /health routes using only Node's http module.

Deliverable: A tiny native Node HTTP server.

Day 10: Express Setup

Learn: Learn why Express is commonly used, how app.listen works, request/response objects, and basic route handlers.

Practice: Create an Express server with GET /health and GET /api/products.

Deliverable: First Express API.

Day 11: Routing and Controllers

Learn: Learn route parameters, query parameters, router files, controllers, and separation of concerns.

Practice: Create /api/products and /api/products/:id routes and move handlers into controllers.

Deliverable: Organized API routes.

Day 12: Middleware

Learn: Understand middleware execution order, next(), built-in middleware, custom middleware, and request logging.

Practice: Create logger, JSON parser, and simple request-timing middleware.

Deliverable: Middleware pipeline.

Day 13: Validation and Error Handling

Learn: Learn input validation, safe error messages, HTTP status codes, async errors, and centralized error middleware.

Practice: Validate a create-product request. Return consistent error JSON.

Deliverable: Validated API with centralized errors.

Day 14: REST API Review

Learn: Review REST conventions, resource naming, status codes, CRUD flow, and API testing.

Practice: Build an in-memory Products CRUD API with GET, POST, PATCH, and DELETE.

Deliverable: Week 2 mini-project: Products API.

REST API Reference

A typical resource might use:

```
GET /api/products -> list products
GET /api/products/:id -> get one
POST /api/products -> create
PATCH /api/products/:id -> update
DELETE /api/products/:id -> delete
```

Useful status codes

• 200 OK — successful read/update.

• 201 Created — resource created.

• 204 No Content — successful operation with no response body.

• 400 Bad Request — malformed or invalid client input.

• 401 Unauthorized — authentication is required or invalid.

• 403 Forbidden — authenticated client lacks permission.

• 404 Not Found — requested resource does not exist.

• 409 Conflict — request conflicts with current state.

• 500 Internal Server Error — unexpected server-side failure.

Consistent error shape

```
{
"success": false,
"message": "Validation failed",
"errors": [
{ "field": "email", "message": "Invalid email" }
]
}
```

Week 3 — Database, Authentication and Security

Day 15: Database Fundamentals

Learn: Learn why applications use databases, tables/collections, records/documents, primary keys, relationships, indexes, and migrations conceptually.

Practice: Choose MongoDB or PostgreSQL for the capstone. Sketch users, products/courses, and enrollments.

Deliverable: A basic data model diagram.

Day 16: Connect Node to a Database

Learn: Learn connection strings, environment variables, connection lifecycle, data-access layers, and handling connection failures.

Practice: Connect your Express app to your chosen database and create one model/table.

Deliverable: Database-connected API.

Day 17: CRUD with Persistence

Learn: Replace in-memory data with persistent storage. Learn create/read/update/delete and basic filtering.

Practice: Implement product/course CRUD against the database.

Deliverable: Persistent CRUD endpoints.

Day 18: Data Relationships

Learn: Learn one-to-one, one-to-many, and many-to-many relationships. Understand foreign keys or references.

Practice: Model users and courses with an enrollment relationship.

Deliverable: Working enrollment data model.

Day 19: Authentication Basics

Learn: Learn identity vs permissions, password hashing, sessions, tokens/JWT concepts, login flow, and protected routes.

Practice: Implement registration and login using a secure password-hashing library. Protect one endpoint.

Deliverable: Working authentication flow.

Day 20: Authorization and Security

Learn: Learn authorization checks, least privilege, CORS, secure headers, rate limiting concepts, input validation, secrets, and avoiding sensitive data leakage.

Practice: Add a protected admin-only or instructor-only route and verify unauthorized access is rejected.

Deliverable: Protected role-aware endpoint.

Day 21: Security Review and API Documentation

Learn: Review authentication, authorization, validation, error handling, environment variables, and API documentation.

Practice: Document API endpoints with request/response examples and test positive/negative cases.

Deliverable: Week 3 security checklist + API documentation.

Week 4 — Testing, Architecture and Production

Day 22: Project Structure and Architecture

Learn: Learn practical layering: routes, controllers, services, repositories/models, middleware, config, utilities, and validation.

Practice: Refactor the capstone into clear modules without changing behavior.

Deliverable: Maintainable backend structure.

Day 23: Testing Fundamentals

Learn: Learn unit vs integration tests, test isolation, assertions, mocking concepts, and testing HTTP endpoints.

Practice: Write tests for a service and at least three API endpoints.

Deliverable: Initial automated test suite.

Day 24: Logging and Debugging

Learn: Learn structured logging concepts, useful error context, debugging with breakpoints, and avoiding console noise in production.

Practice: Add consistent request/error logging and debug one intentionally introduced bug.

Deliverable: Useful logs and debugging notes.

Day 25: Environment Configuration

Learn: Learn .env files, configuration modules, validation of required variables, and secret management principles.

Practice: Move database URL, auth secret, and port into environment variables.

Deliverable: Environment-based configuration.

Day 26: Performance and Scalability Basics

Learn: Understand async I/O, avoiding event-loop blocking, pagination, caching concepts, database indexes, and payload size.

Practice: Add pagination to one list endpoint and measure response behavior with sample data.

Deliverable: Paginated endpoint.

Day 27: Deployment Concepts

Learn: Learn build/start commands, production environment variables, process management concepts, health checks, and deployment platforms.

Practice: Prepare the project for deployment and write a deployment checklist.

Deliverable: Deployment-ready repository.

Day 28: Capstone Build Day

Learn: Combine routing, middleware, validation, database, authentication, authorization, error handling, and tests.

Practice: Complete the main API flows and test them end-to-end.

Deliverable: Functional capstone backend.

Day 29: Capstone Polish

Learn: Improve README, API documentation, validation messages, error responses, tests, logs, and project structure.

Practice: Fix defects and remove unused code. Verify all endpoints with Postman.

Deliverable: Release candidate.

Day 30: Final Review and Interview Preparation

Learn: Review Node fundamentals, Express, HTTP, async behavior, databases, security, testing, and deployment.

Practice: Demo the project from scratch and answer the interview questions in this PDF without notes.

Deliverable: Final capstone + personal revision sheet.

Capstone Project — Learning Management REST API

Build a backend for a small online learning platform. The purpose is to combine the month's concepts into one coherent application.

Core resources

• Users — registration, login, profile.

• Courses — title, description, price/status, instructor.

• Lessons — course lessons with ordering.

• Enrollments — user-to-course relationship.

• Progress — lesson completion for an enrolled user.

• Admin/instructor permissions — restrict management endpoints.

Suggested endpoints

```
POST /api/auth/register
POST /api/auth/login
GET /api/users/me

GET /api/courses
GET /api/courses/:id
POST /api/courses
PATCH /api/courses/:id
DELETE /api/courses/:id

POST /api/courses/:id/enroll
GET /api/users/me/enrollments

GET /api/courses/:courseId/lessons
POST /api/courses/:courseId/lessons
PATCH /api/lessons/:id
DELETE /api/lessons/:id

POST /api/lessons/:id/complete
GET /api/users/me/progress
```

Minimum technical requirements

• Express REST API

• Persistent database

• Environment configuration

• Authentication

• Authorization

• Request validation

• Centralized error handling

• Automated tests

• README with setup and API documentation

Practice Exercises

Beginner

- Create a CLI that accepts a name and prints a personalized message.
- Create a JSON file and implement add/read/update/delete operations.
- Build an HTTP health-check endpoint.
- Create an Express API with three GET routes.

Intermediate

- Build a complete Products CRUD API.
- Add validation and centralized error handling.
- Add pagination and filtering.
- Add authentication and one protected route.
- Connect the API to a database.

Challenge

- Add role-based authorization to the capstone.
- Add automated tests for successful and failing requests.
- Add request IDs or structured logs.
- Implement a consistent API response format.
- Deploy the API and document the production URL and environment setup.

Node.js Interview Revision

1. What is Node.js?

A JavaScript runtime that executes JavaScript outside the browser, commonly used for server-side and tooling applications.

2. Is Node.js a programming language?

No. JavaScript is the language; Node.js is a runtime environment.

3. What is npm?

The package manager and registry ecosystem commonly used to install dependencies and run project scripts.

4. What is package.json?

A project manifest containing metadata, scripts, dependencies, and configuration.

5. What is the event loop?

A runtime mechanism that coordinates asynchronous callbacks/tasks so JavaScript can continue processing work without waiting synchronously for many I/O operations.

6. Why use async/await?

It provides readable syntax for working with promises while preserving asynchronous behavior.

7. What is Express?

A lightweight web framework for Node.js commonly used to build HTTP servers and REST APIs.

8. What is middleware?

A function that can inspect or modify the request/response and either end the response or pass control to the next middleware.

9. Difference between PUT and PATCH?

PUT is conventionally used for replacing a resource representation; PATCH is conventionally used for partial updates.

10. What is REST?

An architectural style for designing networked resources around HTTP semantics and stateless interactions.

11. What is authentication?

Verifying who a user or client is.

12. What is authorization?

Determining what an authenticated user or client is allowed to do.

13. Why hash passwords?

Passwords should not be stored in plaintext; secure password hashing makes stolen database values substantially harder to use directly.

14. What are environment variables used for?

Configuration that should vary by environment, including ports, database URLs, and secrets.

15. What is CORS?

A browser security mechanism controlling which origins can make certain cross-origin requests.

16. Why validate input?

To reject malformed or unsafe data and keep application behavior predictable.

17. What is centralized error handling?

A consistent mechanism for converting application errors into appropriate HTTP responses and logs.

18. Unit test vs integration test?

A unit test focuses on a small isolated unit; an integration test checks interactions between components such as routes and a database.

19. Why avoid blocking the event loop?

Long synchronous work can prevent other requests and callbacks from being processed promptly.

20. What makes a Node API production-ready?

Reliable error handling, validation, authentication/authorization, secure configuration, tests, logging/monitoring, database reliability, performance considerations, and deployment configuration.

Quick Reference Cheat Sheet

Common commands

```
node --version
npm --version
npm init -y
npm install express
npm install -D nodemon
npm run dev
npm start
```

Common Express pattern

```
import express from "express";

const app = express();

app.use(express.json());

app.get("/api/health", (req, res) => {
res.json({ status: "ok" });
});

app.listen(3000, () => {
console.log("Server running on port 3000");
});
```

Recommended project structure

```
src/
app.js
server.js
config/
routes/
controllers/
services/
models/
middleware/
validators/
utils/
tests/
.env
.gitignore
package.json
README.md
```

Final checklist

- I can create a Node project from scratch.
- I understand npm and package.json.
- I can use modules and asynchronous JavaScript.
- I can create an Express REST API.
- I can use routes and middleware.
- I can validate input and handle errors.
- I can connect to a database and implement CRUD.
- I understand authentication and authorization.
- I can write basic automated tests.
- I can prepare and document a Node API for deployment.